TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

Professur
Betriebssysteme

Übung 4 – Arrays, Tupel und Structs

Algorithmen und Programmierung

Martin Richter, Maik Benndorf, Bastian Felix Bachmann, Jens Pönisch

Aufgabe 1 (Fibonacci)

Erstellen Sie eine Funktion `fib_arr()-> [u32; 20]`, welche das Array `arr` mit den ersten 20 Fibonacci-Zahlen befüllt.

Aufgabe 2 (Spiegeln)

Erstellen Sie eine Funktion `mirror(arr: [u32; 10])-> [u32; 10]`, welche das Array `arr` gespiegelt zurückgibt.

Aufgabe 3 (Matrix)

(a) Erstellen Sie eine Funktion `show_mat`, welche eine Matrix (ein zweidimensionales Array) auf der Kommandozeile ausgibt. Wie lässt sich ein zweidimensionales Array darstellen? Wie muss die Signatur der Funktion aussehen?

(b) Erstellen Sie eine Funktion `show_mat_transpose`, welche eine Matrix (ein zweidimensionales Array) auf der Kommandozeile transponiert ausgibt.

(c) Erstellen Sie eine Funktion `mat_mul`, welche das Produkt zweier Matrizen zurückgibt.

Aufgabe 4 (Tupel)

Erstellen Sie eine Funktion `cross(v0: (f32, f32, f32), v1: (f32, f32, f32))-> (f32 , f32, f32)`, welche das Kreuzprodukt von zwei $3D$-Vektoren (in Form von Tupeln) bildet und das Ergebnis als Tupel zurückgibt.

Aufgabe 5 (Structs)

Gegeben sei folgendes Struct `Person`:

```
struct Person {
    age: u8,
    name: [char; 10],
}
```

Die Namen der Personen werden in `[char; 10]`-Arrays verwaltet. Falls Namen weniger als 10 Buchstaben enthalten, wird das restliche Array mit `_` (Unterstrichen) aufgefüllt. (Wie man das schöner machen kann, lernen Sie noch im Laufe des Semesters kennen).

(a) Erstellen Sie eine Funktion `avg_age(persons: [Person; 10])-> f32`, welche für ein Array von Personen das durchschnittliche Alter zurückgibt.

(b) Erstellen Sie eine `print`-Methode für das Struct `Person` (innerhalb des Struct-eigenen Traits).

(c) Erstellen Sie eine `longer_name`-Methode für das Struct `Person` (innerhalb des Struct-eigenen Traits), welche die Person mit dem längeren Namen zurückgibt.